TraceBack

IIT Kanpur Lost & Found Portal

Report it. Hand it in.

Get it back, to the right person.

CS455 Software Engineering · 2026-27
Group 4 · Prof. Priyanka Bagade

Live: `16-176-186-149.sslip.io`

Code: `github.com/cs455project/campus-lost-and-found`

Team and responsibilities

Ankit Agarwal	Foundation, OTP sign-in, roles, profiles, listings
Bhanu Pratap	Lost and found reports, photos, search
Ankit Kumar Dubey	Claims, approval rules, handover
Dev Sachan	Guard custody, item page, app shell
Soumyaranjan Sahu	Admin, jobs, email, audit; D3 testing, AWS deploy
Rohit Yadav	Agentic AI: matching agent and chat assistant

The problem and what we set out to do

Problem

Lost items on campus are chased through WhatsApp groups and notice boards. Found items sit in hostels and offices where no one can search them, and they often go to whoever asks first, with no proof of ownership.

Objectives

- One place to report and search lost and found items
- A verified chain of custody at SIS security offices
- Handover only after a real ownership check
- AI that suggests matches but never decides
- Privacy by design, within free-tier cost

User roles

Student	report, search, claim with private proof
SIS Guard	receive, hold, review claims, hand over
Admin	manage guards and admins, read audit logs

Key features

- Reports with photos; search with filters
- AI matches with reasons; the owner is emailed
- **Ask TraceBack**: a chat assistant for students
- Found ID card → owner emailed (on-server OCR)
- Custody history: who held it, where, when
- Claims with private proof, guard review, ID-checked handover

Requirements engineering and how they evolved

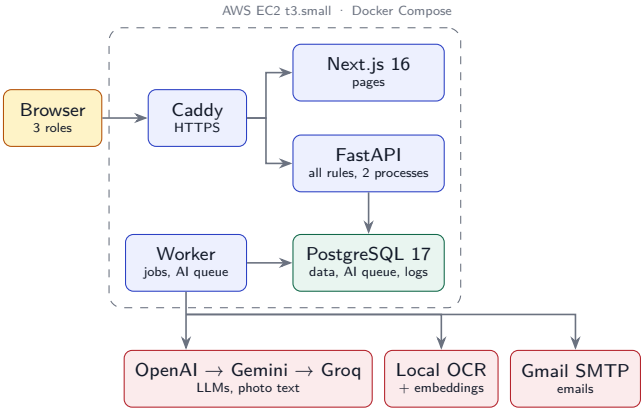
How we captured them

- Stakeholders: students, SIS guards, admin; use cases and a literature review (D1)
- **28 user stories** in Jira, SCRUM-10 to 57, with acceptance criteria
- 11 functional groups (AUTH, REP, SRCH, CUS, CLM, CON, AI, ADM …)
- Non-functional: security, privacy, concurrency, performance, reliability, AI cost
- Agent behaviour as requirements: autonomous, bounded, human-only, prohibited

19 recorded changes; the ones that mattered most

Change	Reason
Every handover goes through a guard; direct finder handover dropped	With no third party, anyone could claim a phone
Item chat dropped	It leaks the details used to verify ownership
ID-card owner alert by local OCR added	IDs returned fast, photo never shared
AI called only when unsure, 500 calls a day	Stay inside free tiers
Verification score, visit slots descoped (D3)	Guard review + ID check already verify; time went to testing

System architecture and design patterns



How the parts interact

- Only Caddy is public: it serves the pages and forwards `/api` to FastAPI; the API and database have no public port
- The API owns every rule and only *queues* slow work
- The worker calls AI and email, so no page waits on them

Design patterns

- **Layers:** routers → services → models
- **State machines** for item and claim status
- **Producer–consumer queue** with `SKIP LOCKED`
- **Strategy:** rule-based or agent matcher
- **Fallback chain:** OpenAI → Gemini → Groq → rules
- **Append-only event logs** for custody and claims

Engineering decisions, workflow and traceability

Decisions that shaped the system

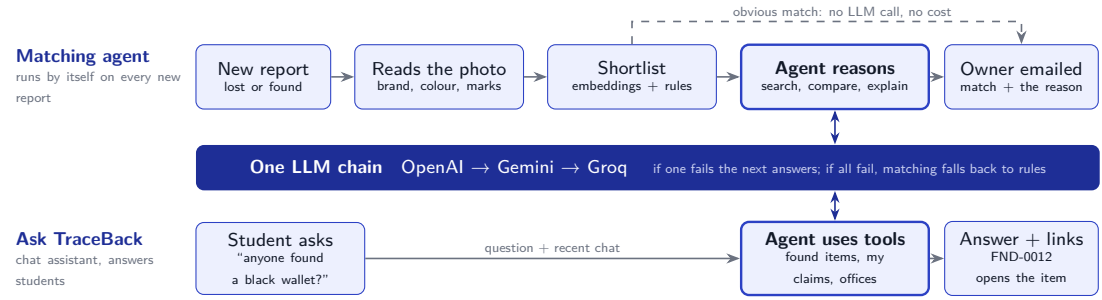
Double allocation	Row locks (item, then claim) *plus* a partial unique index
Sign-in	One-time email code, stored hashed; server-side session in an HttpOnly cookie, not a JWT
AI	The model proposes; code checks ids, dates, thresholds
Privacy	ID-card photos never leave our server
Hosting	One EC2 with Compose, not Kubernetes: fits the load and the budget

Jira → GitHub workflow

18 story PRs (#11 to #28), the key in every branch, commit and PR · 36 reviews, 6 per member · CODEOWNERS and branch protection · D3 fixes on `d3/integration`

One requirement, traced end to end

Agentic AI: two agents, one brain, firm guardrails



Guardrails: the AI suggests, people decide

Does on its own
rank matches, widen a search, explain why, answer questions

Only within limits
email an owner: score $\geq$ 0.7, item held at an office, only once

Only a guard
approve or reject a claim, hand over, move custody

Never
see names, emails, phone or roll numbers; run SQL; change a status

Read-only tools · code checks every id and date the model returns · found ID cards are read only on our server · daily and per-student limits · every run logged, no chat text stored

Testing, debugging, security, performance and deployment

479
unit + integration tests

97.8%
of lines, 91.2% of branches

15/15
browser system tests

23/23
attacks stopped; 2 audits clean

14 × 40
concurrency races, all pass

10/10
failures recover by themselves

Debugging: 25 defects found, all 25 fixed

Worst (critical): the API locked up at 25 users. A profiler showed 40 threads waiting on 15 database connections. We capped requests in flight and sized the pool. **Now:** 100 users, 0 errors, about 250 requests/s (was 71); 0.8 MB per page (was 18 MB).

Others: double handover in a race (row locks); GPS left in photos (stripped); crashed AI jobs lost (re-queued); bare 500 when the database was down (clear 503).

Security

STRIDE model of 22 threats, each with a control in the code. What remains is operational: key rotation and MFA.

Deployment

AWS EC2 t3.small (Sydney), 5 Docker containers, Caddy auto-HTTPS, one-command deploy; about US$25 a month, paid by free credits.

Lessons

- Test concurrency and load from sprint one: they found our worst bugs
- Measure, don't assume: the bottleneck wasn't where we guessed
- Leaving data out beats inventing it